

LAB: Digital Image Processing

Lab on Image restoration

Objective:

- I. Understand image noise and its types.**
- II. Learn to add artificial noise to images.**
- III. Learn to restore images using spatial filters (Mean, Median, Gaussian).**
- IV. Compare filter performance visually and quantitatively.**

LAB: Digital Image Processing

Lab on Image restoration

What is Image Restoration?

Image restoration aims to recover the original image from a **noisy or degraded version** using mathematical or filtering techniques.

LAB: Digital Image Processing

Lab on Image restoration

Types of Noise

Gaussian Noise: Random variations with a normal distribution.

Salt-and-Pepper Noise: Random white and black pixels.

Speckle Noise: Multiplicative noise common in radar or ultrasound images.

LAB: Digital Image Processing

Lab on Image restoration

Filter	Description	Use
Mean Filter	Averages pixel values in a window	Smooths Gaussian noise
Median Filter	Takes the median in a window	Removes salt-and-pepper noise
Gaussian Filter	Weighted average with Gaussian kernel	Reduces Gaussian noise smoothly

LAB: Digital Image Processing

Lab on Image restoration

Noise Type	Common Function / Method	Description
Gaussian Noise	<code>numpy.random.normal(mean, sigma, img.shape)</code>	Adds Gaussian (normal) noise to the image. Can adjust mean and standard deviation (sigma) to control intensity.
Salt & Pepper Noise	Custom function using <code>np.random.randint</code>	Randomly sets some pixels to maximum (255 – salt) or minimum (0 – pepper). Not directly available in OpenCV, usually implemented manually.
Poisson Noise	<code>numpy.random.poisson(lam, img.shape)</code>	Adds noise according to Poisson distribution, simulating photon shot noise.
Speckle Noise	<code>img + img * np.random.randn(*img.shape)</code>	Multiplicative noise, common in radar or ultrasound images.

LAB: Digital Image Processing

Lab on Image restoration

Filter	OpenCV / NumPy Function	Description
Mean (Average) Filter	<code>cv2.blur(img, (k, k))</code>	Smoothens the image by replacing each pixel with the average of its neighbors. Good for Gaussian noise but blurs edges.
Weighted Mean (Gaussian) Filter	<code>cv2.GaussianBlur(img, (k, k), sigmaX)</code>	Applies Gaussian kernel for smoothing. Preserves edges better than simple mean.
Median Filter	<code>cv2.medianBlur(img, k)</code>	Replaces each pixel with median of its neighbors. Very effective for Salt & Pepper noise.
Bilateral Filter	<code>cv2.bilateralFilter(img, d, sigmaColor, sigmaSpace)</code>	Edge-preserving smoothing; good for Gaussian noise while keeping edges sharp.
Laplacian / Unsharp Mask (Restoration)	<code>cv2.Laplacian(img, cv2.CV_64F)</code>	Used for edge enhancement and deblurring.

Required Libraries

```
import cv2  
import numpy as np  
import matplotlib.pyplot as plt
```

Step # 1

```
# Load grayscale image  
img = cv2.imread('lena.jpg', cv2.IMREAD_GRAYSCALE)  
  
# Display using matplotlib  
plt.figure(figsize=(5,5))  
plt.imshow(img, cmap='gray')  
plt.title('Original Image')  
plt.axis('off')  
plt.show()
```

Original Image



Step # 2

```
# Add Gaussian noise  
row, col = img.shape  
mean = 0  
sigma = 20  
gauss = np.random.normal(mean, sigma, (row, col))  
noisy_gauss = img + gauss  
noisy_gauss = np.clip(noisy_gauss, 0, 255).astype(np.uint8)  
  
plt.imshow(noisy_gauss, cmap='gray')  
plt.title('Gaussian Noisy Image')  
plt.axis('off')  
plt.show()
```

Gaussian Noisy Image



Step # 2

```
[4]: # Add Salt and Pepper Noise
noisy_sp = np.copy(img)
prob = 0.02 # 2% pixels affected
for i in range(row):
    for j in range(col):
        rdn = np.random.random()
        if rdn < prob/2:
            noisy_sp[i][j] = 0 # pepper
        elif rdn > 1 - prob/2:
            noisy_sp[i][j] = 255 # salt

plt.imshow(noisy_sp, cmap='gray')
plt.title('Salt & Pepper Noisy Image')
plt.axis('off')
plt.show()
```

Salt & Pepper Noisy Image



Filtering Techniques (Noise Removal)

(a) Mean (Average) Filter

```
[5]: mean_filtered = cv2.blur(noisy_gauss, (3,3))  
plt.imshow(mean_filtered, cmap='gray')  
plt.title('Mean Filtered Image')  
plt.axis('off')  
plt.show()
```

Mean Filtered Image



Filtering Techniques (Noise Removal)

(b) Median Filter

```
[6]: median_filtered = cv2.medianBlur(noisy_sp, 3)
plt.imshow(median_filtered, cmap='gray')
plt.title('Median Filtered Image')
plt.axis('off')
plt.show()
```

Median Filtered Image



Filtering Techniques (Noise Removal)

(c) Gaussian Filter

```
[7]: gaussian_filtered = cv2.GaussianBlur(noisy_gauss, (5,5), 1)  
plt.imshow(gaussian_filtered, cmap='gray')  
plt.title('Gaussian Filtered Image')  
plt.axis('off')  
plt.show()
```

Gaussian Filtered Image



Filtering Techniques (Noise Removal)

Comparison

```
[8]: titles = ['Original', 'Gaussian Noise', 'Mean Filter', 'Gaussian Filter']  
     images = [img, noisy_gauss, mean_filtered, gaussian_filtered]  
  
     plt.figure(figsize=(10,5))  
     for i in range(4):  
         plt.subplot(1,4,i+1)  
         plt.imshow(images[i], cmap='gray')  
         plt.title(titles[i])  
         plt.axis('off')  
     plt.show()
```

Original



Gaussian Noise



Mean Filter



Gaussian Filter



Filtering Techniques (Noise Removal)

Tasks for the students

	Task	Hint
1	Add uniform noise and try removing it with mean filter with two different images	Use <code>np.random.uniform()</code>
2	Change Gaussian $\sigma = 10, 30, 50$ and compare results	Observe the changes
3	Apply median filter with size 5×5 on salt-pepper noise	Use <code>cv2.medianBlur()</code>
4	Increase the probability for salt & pepper noise and observe the results	Observe the performance of filter
5	Try bilateral filter for Gaussian noise	<code>cv2.bilateralFilter()</code>
7	Display all filtered images in one subplot	Use matplotlib subplot